

Credit Card Fraud Detection

End-to-End Machine Learning Project — GitHub Project Summary

Built by Nader Fawzy

A complete fraud-detection workflow covering data integration, quality auditing, feature engineering, imbalanced classification, model comparison, hyperparameter search, decision-threshold optimization, evaluation, model serialization, and a Streamlit inference interface.

1. Project Objective

The project builds a binary classification system for identifying potentially fraudulent credit-card transactions. The target variable is `Is Fraud?`, converted to a binary target where Yes = 1 and No = 0. The main challenge is extreme class imbalance: fraudulent transactions represent only about 0.12% of the modeled transactions.

2. Data Sources and Integration

The notebook loads transaction, card, user, and an additional credit-card transaction source. Transaction records are enriched with customer and card attributes through a many-to-one merge using (User, Card) against (User, CARD INDEX). The redundant CARD INDEX field is removed after integration.

- The full integrated dataset previously audited in the project contains approximately 24.39 million transaction rows; the notebook then creates a 2,000,000-row sample preserving the original fraud ratio for the modeling workflow.
- The modeling sample is shuffled with a fixed random seed and checked for duplicates.
- Card/user consistency and join integrity were explicitly inspected before modeling.

3. Data Quality and Preprocessing

Financial and transactional variables

- Currency-formatted values such as Amount and Credit Limit are converted from strings to numeric values.
- Negative transaction amounts are retained rather than converted to absolute values, preserving refund/return information.
- Zero and negative amounts were investigated against transaction type and MCC instead of being blindly removed.

Date and time processing

- Year, Month, Day, and Time are combined into a datetime representation.
- Derived features include hour, day of week, day of month, month, weekend indicator, and cyclic sine/cosine representations for hour and month.
- Cyclic encoding allows adjacent times such as 23:00 and 00:00 to remain close in feature space.

Error information

- The multi-label Errors? field was decomposed into binary indicators including bad PIN, insufficient balance, technical glitch, bad card number, bad CVV, bad expiration, bad ZIP code, plus an error count.
- For the final modeling matrix, the raw Errors? field and years_since_pin_change were explicitly removed during leakage/feature validation.

Geographic and transaction-context features

- same_state, same_city, same_zip, is_online, and is_international were engineered from customer and merchant information.
- Account age and months until card expiry were derived from transaction time and account/card dates.

4. High-Cardinality Merchant Handling

Merchant Name contains 100,343 unique merchant identifiers. Its frequency distribution is highly skewed: the median merchant appears only 4 times, while the most frequent merchant appears more than 1.13 million times. Treating Merchant Name as a raw numeric value would incorrectly impose an ordinal relationship on merchant IDs.

- Merchant Name is excluded from direct numeric modeling.
- Merchant frequency is calculated from the training split only and mapped to validation/test data.
- An is_rare_merchant indicator is derived using a frequency threshold of 5.
- MCC is one-hot encoded to represent merchant category rather than merchant ID magnitude.

5. Feature Encoding and Leakage Controls

- One-hot encoding is fitted on training data for Use Chip, Gender, Card Brand, Card Type, and Has Chip.
- MCC is one-hot encoded with handle_unknown='ignore'.
- Merchant frequency is learned from X_train and applied to validation/test; unseen merchants receive frequency 0.
- Identifier fields such as User and Card, raw location identifiers, raw dates/times, target columns, and other forbidden fields are checked and excluded from the final feature matrix.
- The final validation function verifies missing values, non-numeric columns, and forbidden columns before model training.

6. Train / Validation / Test Strategy

The notebook uses stratified random splitting to preserve the rare fraud class across all subsets: 60% training, 20% validation, and 20% test. The resulting validation and test sets each contain 488 fraud transactions out of 400,000 rows.

Because the project uses random stratification rather than a chronological split, the evaluation should be interpreted as an experimental benchmark for this dataset rather than a direct simulation of future production deployment.

7. Models Evaluated

Model	Handling / configuration	Validation PR-AUC
Logistic Regression	StandardScaler + class_weight='balanced'	0.1187
Random Forest	Balanced class weights; depth/leaf tuning	0.0752 (v2)

XGBoost	scale_pos_weight + subsampling + regularization	0.2800 (initial)
Tuned XGBoost	RandomizedSearchCV, 25 candidates, 3-fold CV	0.7531

8. XGBoost Optimization

Because fraud is extremely rare, the project uses Average Precision / PR-AUC as the main search metric. The positive-class weight is computed as the ratio of negative to positive training examples, producing `scale_pos_weight` ≈ 818.67 .

- RandomizedSearchCV evaluates 25 parameter combinations using 3-fold cross-validation.
- Search space includes `max_depth`, `learning_rate`, `n_estimators`, `min_child_weight`, `subsample`, `colsample_bytree`, and `scale_pos_weight`.
- Best cross-validation PR-AUC reported by the search: 0.7082.
- The selected estimator achieved 0.7531 validation PR-AUC and 1.0000 train PR-AUC.

9. Decision Threshold Optimization

The default probability threshold of 0.50 is not assumed to be optimal. Precision, recall, and F1 are evaluated across thresholds. The selected threshold is 0.634, based on the validation analysis.

- Validation at the selected operating point: Precision ≈ 0.9201 , Recall ≈ 0.6373 , F1 ≈ 0.7497 .

This illustrates the operational trade-off: increasing the threshold generally improves precision while reducing recall. In a financial fraud system, the threshold should ultimately be chosen according to the relative cost of false positives and false negatives and the institution's review capacity.

10. Final Test Results

Metric	Test result
PR-AUC / Average Precision	0.7587
ROC-AUC	0.9896
Fraud Precision	0.9189
Fraud Recall	0.6270
Fraud F1-score	0.7454
Accuracy	0.9995

Confusion matrix at threshold 0.634: TN = 399,485; FP = 27; FN = 182; TP = 306. The model therefore identifies 306 of 488 fraud transactions in the held-out test set, while 27 legitimate transactions are classified as fraud.

11. Feature Importance

The final XGBoost analysis reports the following among the most influential inspected features: `same_state`, `MCC_4784`, `same_city`, `is_rare_merchant`, `merchant_frequency`, `same_zip`, `MCC_4121`, `Use Chip_Chip Transaction`, `is_international`, and `is_online`. Feature importance is model-specific and should not be interpreted as causal influence.

12. Deployment

The trained model is serialized as a Joblib artifact and integrated into a Streamlit application. The deployment layer exposes the preprocessing/model workflow through an interactive interface rather

than requiring notebook execution.

- Model artifact: fraud_xgboost_pipeline.joblib
- Interactive application: Streamlit
- The production-facing decision uses the selected probability threshold rather than relying blindly on 0.50.

13. End-to-End Workflow

Raw transaction/card/user data → Data validation → Merge → Cleaning → Feature engineering → Train/validation/test split → Encoding → Class-imbalance handling → Model comparison → XGBoost hyperparameter search → Threshold optimization → Test evaluation → Model serialization → Streamlit inference.

14. Key Technical Skills Demonstrated

- Python, Pandas, NumPy, Matplotlib, Seaborn
- scikit-learn preprocessing, model training, metrics, and RandomizedSearchCV
- XGBoost for highly imbalanced binary classification
- One-hot encoding and leakage-aware frequency encoding
- Feature engineering for time, geography, merchant behavior, account age, and card status
- PR-AUC, ROC-AUC, Precision, Recall, F1, confusion matrix, and threshold optimization
- Model serialization with Joblib and application deployment with Streamlit

15. Limitations and Next Steps

- The notebook uses a stratified random split; a chronological train/validation/test split would better simulate future fraud detection.
- The model should be monitored for data drift, concept drift, changing fraud patterns, and threshold performance after deployment.
- Historical velocity and behavioral features can further strengthen the system if constructed strictly from information available before each transaction.
- Graph/network features can be added to detect coordinated accounts and money-mule patterns.
- Threshold selection can be upgraded from F1 optimization to an explicit business-cost function incorporating false-positive and false-negative costs.

Project summary generated from the provided froad1.ipynb and the stated deployed pipeline/Streamlit implementation. Reported metrics are taken from the notebook outputs.